

GSoC'21 Proposal - CircuitVerse

Internationalization - I18n

Personal Details

Name: Pavan Sanjay Joshi

Course: Computer Engineering

University: DR.Babasaheb Ambedkar Technological University,
Raigad, Maharashtra

Current Year: 2nd

Email: pavanjoshi2050@gmail.com

Github: <https://github.com/pavanjoshi914>

LinkedIn: <https://www.linkedin.com/in/pavan-joshi-177873192>

Phone:

Current Country: India

Resume: [link](#)

About Me

I'm Pavan Joshi, an undergraduate second year computer engineering student currently enrolled in Dr. Babasaheb Ambedkar Technological University, Raigad, Maharashtra, India.

I received introduction to computer programming throughout the time of my secondary school that attended me to believe that coding is the language of the future, as a consequence I decided to take computer engineering as my core branch and this formed me passionate about coding.

For the past one year, I have been practicing competitive programming using C++ as a core language. Competitive programming assisted me a lot to build my logic and take my problem solving skills at another level. It also assisted me to convert my ideas into code.

Initially I have been working in MERN stack as a Full Stack Developer. I have an adequate edge on different development concepts such as Git, Github, Node.js, Express.js, RESTful API, Database fundamentals, Deployment, Authentication and Security, React.js, JavaScript ES6 and many more.

For the past 6 months, I have been working on Ruby on Rails framework and JavaScript at CircuitVerse. I have an adequate edge on Rails MVC framework, gems, Testing in Rspec, behaviour driven development etc. Now I am a proficient developer in MVC based frameworks.

Motivation

My biggest motivation to apply for this project is the current situation of pandemic and the Importance that Online learning resources and tools nowadays have.

The 21st century is the century of digital revolution and the Internet is the most common tool for learners and students. Any information available on the internet is always one click away from the user, however language difference is the only limitation because of which people can't make full use of resources available on the internet. This is why globalization of the Internet requires content that any one can understand.

As per a recent survey conveyed by [statista](#) it is conveyed that almost 76.9% of global users like to surf on the internet in their native language. This provides me with excellent motivation to apply for the I18n project.

I believe that Internationalization will let more people know about the CircuitVerse platform and get connected with the community. I believe that Internationalization is the best way to represent this awesome product to International clients, learners and teachers.

Major Interest

According to wikimedia 180,000 open source projects are serving people all around the world, thus open source has yielded a tremendous growth in the software industry. This provides me with a keen interest to contribute to open source projects.

In my opinion the best way to be a good programmer is applying knowledge and set of skills in practical scenarios. I believe open source acts as a huge platform to learn and sharpen my knowledge in various technologies and programming principles.

My experience with open source is small, but I received a great exposure to open source workflow during my initial contributions to CircuitVerse platform. Also I am prepared to learn a lot more about open source and brush up my skills during the GSoC period.

Commitment

1. Are you planning any vacations during the GSoC period?

No, I will be allocating my summer time to my GSoC project.

2. How many classes are you taking during the GSoC period?

Tentative end of classes for the current semester in my college will be 22 June. except the first week of GSoC there will rarely be any classes.

3. Do you have any other employment during the GSoC period?

No, I don't have any other employment during the GSoC period.

4. How many hours per week do you expect to work on the project and what hours do you tend to work?

I will be coding 4 hours a day Monday to Friday. I will be focusing on testing, code quality improvements on weekends having 30+ working hours a week.

Also, I will be available to work full time when required.

Contributions so far

Patches submitted:

SR	Pull Request	Status	Link To PR	Description
1	Group Invitation System	Merged	Feature#1997	This feature provides an easy and handy way to add members to a group via sharing a link which is expirable at a period of one week.
2	Generate Digital Circuit via Boolean Function	Merged	Feature#1971	This feature takes a boolean function as an input from the user and generates the circuit instantly on the simulator canvas.
3	Site Wide Announcement (From Scratch)	Merged	Feature#1804	This feature provides an easy way to make announcements on CircuitVerse Website.
4	Responsive ToolTips for Project Names	Merged	Feature#1681	This feature implements responsive tooltips to represent long names of projects and maintain consistent layout of all circuit cards.
5	Major Fixes in Simulator NavBar	Merged	Fix#1878 Fix#1936	This fixes major issues present in the navbar of the simulator and makes it responsive for smaller screens.
6	Improvements in Site Wide Announcement Feature	Merged	Fix#1835	This fixes minor bugs present in the announcement banner and implements Wysiwyg text editor (trumbowyg) to create announcements easily.
7	Improvements in Trumbowyg Text Editor	Merged	Feature#1844	Trumbowyg has a limited number of features, this feature aims at increasing the number of features and extensions in Trumbowyg.

8	Implementing Image Upload Plugin in Trumbowyg	Merged	Feature#1932	Implementing Image upload plugin in Trumbowyg,using IMGUR api as image host.
9	Give Default Font to Website	Merged	Feature#1662	adding default font 'Inter' to CircuitVerse website.
10	Support for Different font families in Trumbowyg	Merged	Feature#1953	Introducing a new font family plugin to support text in 8 different fonts families.
11	Fix in Copy to Clipboard Button	Merged	Fix#2056	correction in naming of class name which failed JavaScript code
12	Fix in CheckBox Container	Merged	Fix#2029	Decreasing container area of checkbox so that tick can be done only through checkbox and its label.
13	Improving UI of User Search Page	Ready to merge	Feature#2030	Giving effective layout and making it in sync with the project search page.
14	Integration of Zoom Button in Simulator Zoom Slider	Merged	Feature#1972	Introducing zoom in and zoom out in the simulator via buttons
15	Signup Form Validation	Ready to merge	Feature#1948	Applying proper validations to sign up form.

Issues Raised and Feature Requests:

SR	Issue	Link
1	Sign Up Form Validation	Issue#1640
2	Major Bugs in Simulator NavBar	Issue#1693
3	Browser compatibility of Simulator UI	Issue#1740

4	Integrate Group Invitation Feature in CircuitVerse API	Issue#2057
5	Links Broken in ReadMe	Issue#533

6	Form Restrictions in Mobile App	Issue#85
7	User Search Results gets titled as Projects Search	Issue#1667
8	Improvements In Combinational Analysis Feature	Issue#1915

Other contributions

I am an active member on CircuitVerse slack channel and helped a lot of new contributors to get their local development environment setup.

Along with my programming contributions to CircuitVerse, I have frequently assisted new contributors and encouraged them to bring their contributions with CircuitVerse. I have participated in various discussions and always enthused to solve any type of problem. I have also reviewed other contributor's PR's frequently and given my opinion on it.

Proposal

Overview:

I18n is a process of obtaining all strings out of application and having their translations in the desired language. Ruby on Rails framework (≥ 2.2) provides a basic I18n framework to internationalize Rails applications and can be used to provide optimal support for I18n along with other I18n tools and plugins.

CircuitVerse Simulator is written in application-like JavaScript and bundled using Rails webpacker. Rails I18n can't be used directly in Simulator. however we will be using I18n-Js gem as a middleware to provide I18n support in Simulator.

Interactive-Book is a static site built using the Jekyll framework and has no native support for I18n. However Jekyll plugins can be used for basic I18n support in Interactive-Book.

CircuitVerseDocs is a static site built using DocsifyJs, a static site generator and can be internationalized manually without any native or external plugin support for I18n.

CircuitVerse Mobile-App is a Flutter application and can be internationalized using the Intl package workflow.

Objectives:

Main Objective of this project is Internationalization and localization of all CircuitVerse primary and secondary platforms by creating a flexible I18n infrastructure in such a way that creating, editing, modifying, and managing translations in all platforms can be done in an easier way.

Programming Languages to be used:

Core technologies, I will be working on in different platforms are as follows -

1. Ruby on Rails in CircuitVerse Main Platform
2. JavaScript in CircuitVerse Simulator
3. Jekyll in CircuitVerse Interactive-Book
4. Flutter in CircuitVerse Mobile-App
5. DosifyJs in CircuitVerse User Documentation

Major Tools Used:

- In Ruby on Rails
 - [Rails-I18n](#): General I18n Support in Rails (Already used)
 - [Mobility](#): Model Translation
 - [I18n-tasks](#): As a developer dependency to remove orphaned or missing keys
 - [i18n-spec](#): To check validity for locale files
- In JavaScript
 - Yarn dependencies:
 - [I18n-Js](#): Middleware to export translations on client side for simulator (Already used as a gem)

- [Rails ERB Loader](#): Enables use of ERB in webpacker (required during I18n-js setup with webpacker)
- In Jekyll
 - [Jekyll Polyglot](#): I18n support for Jekyll framework
- In Flutter
 - [flutter_localizations package](#) : I18n support for Flutter Built-In Widgets (Already used)
 - [Intl package](#): I18n support for Flutter and its custom widgets (Already used)
 - [Shared_preferences package](#): To persist user locale (Already used)
- In DocsifyJs
 - No external Support required

Internationalization of CircuitVerse Rails platform (I18n for Ruby on Rails)

Table of Contents:

- [Synopsis](#)
- [Configuration of I18n Module](#)
- [Set, Preserve and Switch Locales](#)
- [Localization Rules](#)
- [Rails Lookup](#)
- [Translation Directory Structure](#)
- [Internationalization of CircuitVerse Rails Platform](#)
- [Localization of Models Via Mobility gem](#)
- [Tests for Integrity of Translation Files](#)

Synopsis:

I18n in Rails aims at abstracting all the static and locale specific strings out of Rails application and form a structure of keyed dictionaries (using .yaml or .rb files) and store them in an efficient directory structure.

Internationalization Includes translation of content strings (static strings) and also non content strings such as date and time formats, months, Active Record model translations, strings popped by a variety of gems (Devise, Paginators), Model translations, notices, flash messages and so-forth.

We will be using already used [Rails-I18n](#) gem which provides general locale data for many languages, interpolation and pluralization of variables, customizable directory structure for efficient key management, public API with **translate** and **localize** methods, lookups, rich set of attribute readers and writers and so forth.

Configuration of I18n Module:

Despite the basic Initialization of the I18n module done in Rails platform, new attribute readers and writers including **translation load path** and **fallbacks** are necessary.

According to load order of the environments in Rails, **application.rb** is loaded before the Rails initializers, as a consequence all the I18n configuration will also be done in **config/application.rb**

Translation File Formats:

Translations in Rails are generally stored in **YAML** or in **Plain Ruby** formats.

YAML are highly readable data serialization files providing data storage in the form of key-value pairs .They are also widely used as configuration files including storage of translation files in the Rails community.

We will be using **YAML** to store localized data in the form of key-value pairs using YAML formats.

Translation Directory Structure and Translation Load Path:

By default the public API of the I18n framework looks for translation files available at the top level of **config/locale** folder.

Managing all locales just using top level directory structure is very inconvenient, as a solution Rails I18n allows us to use **load_path** to structure translation directory according to our needs.

```
config.I18n.load_path += Dir[Rails.root.join('config',  
  'locales', '**', '*.rb,*.yml')]
```

Configuring translation load path using appropriate wildcards leads in search for required files present at top level.

If not found then searching will jump into nested folders for finding required translation files and keys resulting in flexible translation directory structure.

Set, Preserve, and Switch Locales:

Setting locale:

In Rails locales are set up using **I18n.locale** attribute setter.

Even so, basic configuration for setting locale is done using **http accept language header** and through User form, other methods to set locale as can be used ([See](#)).

A locale set using HTTP GET parameter will be an additional helpful method when the CircuitVerse API needs to give a localized version of JSON response in order to translate dynamic content in Mobile App.

Preserving Locale:

We already use mechanisms to store the locale of every user in the **User** model in a separate column **locale**.

Designing Switcher to Switch Locale:

In order to switch languages easily, A site wide language selector will be implemented.

Understanding the acronym of each locale is hard, as a solution we will be using the full language names of the locales in switcher so that users can select the language of their choice easily.

Localization Rules:

Managing keys in an efficient way is an important aspect of internationalization in terms of naming of keys, avoiding duplication of keys and organization of keys.

Rules for localization offer a set of basic rules which can be used during project implementation and by future contributors. Rules for localization will help us to introduce new keys and modify existing one easily.

- Explicit Naming of keys

Rails provides many lookup techniques including lazy lookup which may not help in understanding the purpose of the key completely.

Explicit key names help in understanding the purpose and where it belongs easily and shall be named as **key prefix + key name**

Key prefix shall be named according to the context of the current resource or file name in which key will be used.

Key names shall be named according to the purpose of the key and not according to the file name or resource where it is used.

Current rule helps us in following way -

- Rule helps to find the location of keys easily.
- Rule helps in finding and debugging keys in an easier way.
- Allows the same key to be used in different sections, as a result decreasing collision of keys to a greater extent.

Example

`t("Teachers.main_heading")` - meaningful
`t(".main_heading")` - not much meaningful

- Treating keys as methods

In order to avoid collisions and duplication of keys, keys shall be treated as methods in a particular namespace or in the whole translation file.

Care shall be taken while introducing a new key whether there is a key present with the same name or not.

- Nesting of keys in translation files (Naming for key prefix)

Nesting of keys helps in defining where the key is actually used.

Keys which are repeatedly used throughout the app shall be included in `general` namespace and hence shall have prefix name as `general.*`

Example: `t("general.save")`

Any normal key used to provide translation, Its prefix should start with the `view_name` in which key has to be used.

Current rule ensures a decrease in the number of nesting of keys, providing a meaningful prefix to the key and also helps in finding the location of key easily.

Example:

Consider logix controller having its views in logix folder

```
# logix/teachers.html.erb
```

```
key: t("teachers.key_name")
```

- Style for key naming

Although keys can be named using `snake_case` or `CamelCase` but for long key names `snake_case` is obviously more readable and will

be used for naming of keys.

- Naming of key

Naming of key plays an important role in rails I18n as modification in content must not lead to modification in its corresponding translation key.

The solution is to name the key according to the context or purpose of the key in order to keep the translation key in sync with the content.

This ensures easy modification of content without the need of modifying its translation key.

Naming keys according to the purpose of keys helps in using the same key in different sections of the page resulting in a decrease in the number of key declarations.

- Complexity
Moderate nesting of keys with meaningful namespace shall be done instead of bulky nesting.

Rails Lookup:

As the explicit name of the key represents the meaningful purpose of the key, we will be majorly using basic lookup instead of other Rails lookup with the key name defined explicitly.

Basic lookup:

Basic lookup can be written in different styles as follows and can be used accordingly

- Key defined explicitly in form of string:
`t('Teachers.title')`
- Key defined explicitly with defaultMessage attribute:
`t( { key: 'Teachers.title', defaultMessage: "Main Title" } )`

Similarly Basic lookups can be used for localization functions also.

Translation Directory Structure:

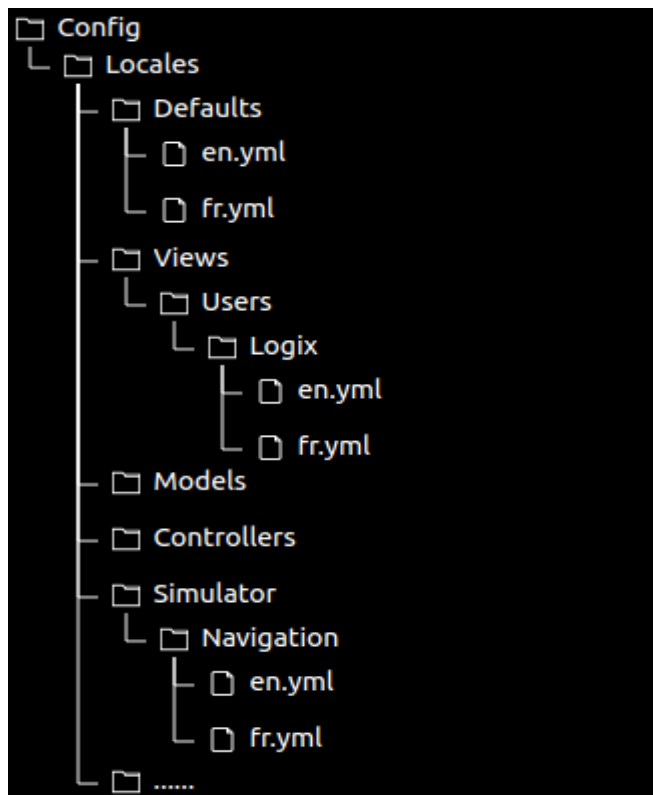
Setting up an appropriate translation path in the translation load path using Ruby glob matchers helps us in customizing and creating flexible translation directory structure.

Translation directory structure will be designed in the same way code files are managed in codebase in order to know the location of keys easily.

General translations repeating throughout the app will be stored in the **Defaults** folder.

Non repeating translations are stored in folders according to the section they belong to in the codebase.

Generally, translation directory structure will be designed as follows -



Internationalization of CircuitVerse Rails Platform:

Although translations will be provided throughout the platform, some of the major Rails section to be localized are as follows -

- Views
- Models
- Forms
- Controllers
- I18n in gems (Devise, Paginator etc)

Views:

Most of the static strings lie in views and strings abstracted and localized are stored in the translation directory structure in `config/locale/views` folder.

Further nesting of folders can be done according to the nesting of views done in the codebase, resulting in more modularity in the translation directory structure.

Models:

Model contains attributes including other static strings and validation error messages.

Backend to store model translations can be designed In many ways, however we will be using `Mobility-gem` for efficient storage and retrieval of translations.

Active Record validation error messages and notices can be localize using single translation file and translations nested in namespaces as follows -

```
errors:
  messages:
    invalid_email_format: is not a valid email
```

Forms:

Rails Form are basically made up of three attributes

- Form Labels
- Form Input Fields
- Form Submit Button

Translations for form strings can be nested using their own attribute names as follows

```
form:
  labels:
    username: 'Username'
  input:
    username: 'Your Username'
  submit: "submit"
```

I18n in CircuitVerse forms includes localization of date and time formats. Rails I18n gem provides rich set of localized date, time formats along with their translations in localized language.

Controllers:

Controllers majorly contain flash messages, notices and plain texts, response messages in controller methods and so forth.

Overall translation key for controllers strings can be named as **controller_name + method_name or resource_name** .

Example -

Response message for successful creation of user in users controller can be represented as - **t('users.create.success_message')**

Strings occurring from gems:

Devise: Devise views are already localized for English language (using [official guide](#)). Similarly translations file will be created for other languages,

Devise will load correct translations according to the currently set locale of the User.

Will-Paginate: output of paginator can be translated by following the official guide for I18n for will-paginate and creating appropriate translations files and keys. ([official guide](#)).

Localization of Models via Mobility gem:

Localization of Rails models includes localizing user generated content or Model attributes present in the database without changing any code logic in views and controllers.

Mobility gem will be used to localize models in an efficient way, Mobility provides a number of backend choices to be used to store translations along with backend independent features such as caching, fallbacks, locale accessors, dirty tracking etc.

Mobility provides 6 types of backends as follows -

- Column Backend
- Table Backend
- Key Value Backend
- Serialized Backend
- .hstore and .jsonb Backend

Provisionally I have selected the **key-value backend** via comparing it with the other backends and it seems more suitable as compared to other backends.

However, choice of using the backend can be changed after in depth analysis of the database and according to the priorities, as each backend has its own set of limitations. ([See here](#))

Tests for Integrity of Translation Files:

Translation files can have typos and the most basic typo is not giving proper language keys on the top of the yaml file.

Typo in the language key can lead to serving of wrong translation file or translation not found errors for different locales.

As a solution Integrity tests will be written which basically loads a particular translation file and checks integrity by matching key definition of file with file definition.

For advanced I18n testing I will be using an [I18n-spec](#) gem.

Internationalization of Simulator (JavaScript)

Table of Contents:

- [Synopsis](#)
- [Configuration of I18n-js](#)
- [Translation Directory Structure](#)
- [Internationalization of Simulator](#)

Synopsis:

CircuitVerse simulator is written in JavaScript and bundled by webpacker in its own pack file named simulator pack and serves Simulator JavaScript files on the client side.

Basic Rails I18n framework cannot be used on app-like Javascript served by webpacker, as a solution we need a middleware which can allow us to provide custom translations on the client side.

we will be using [I18n-js](#) gem as a solution to feed translations in simulator code. I18n-js gem is easy to integrate with webpacker and provides a rich set of features such as Pluralization, Date and Time localization, number localization, locale fallback, variable Interpolation and so forth.

I18n in simulator includes:

- Setup I18n-js with webpacker
- I18n in Simulator interface and components such as navbars, titles, label names, draggable panels, dropdowns, tool tips etc

- I18n in draggable panels and their tag names (Simulator Controls such as properties panel)
- I18n in different Prompts exported from javascript files (eg: combinational analysis prompt)
- I18n in HTML which are appended via JavaScript in UI
- I18n in components which are generated by complex JavaScript logic

Configuration of I18n-js:

We are currently using I18n-js gem and it can be setup via asset pipeline as well as with webpacker.

I18n-js provides middleware to export translation files on the client side, it also allows us to export selective translation files of our choice into multiple numbers of **JS object files**. This means we can export only required simulator translation files easily on the client side excluding translations used for Rails platform.

I18n-js can be added as a yarn dependency to integrate it with webpacker. After a couple of steps of configuration ([see](#)) we can use translation function in the form **I18n.t('key_name')** to provide translation in Simulator on JavaScript side.

However we have to tell I18n-js explicitly about default locale and current value of locale so that appropriate translations are loaded according to current active language of the site.

```
<script type="text/javascript">
  I18n.defaultLocale = "<%= I18n.default_locale %>";
  I18n.locale = "<%= I18n.locale %>";
</script>
```

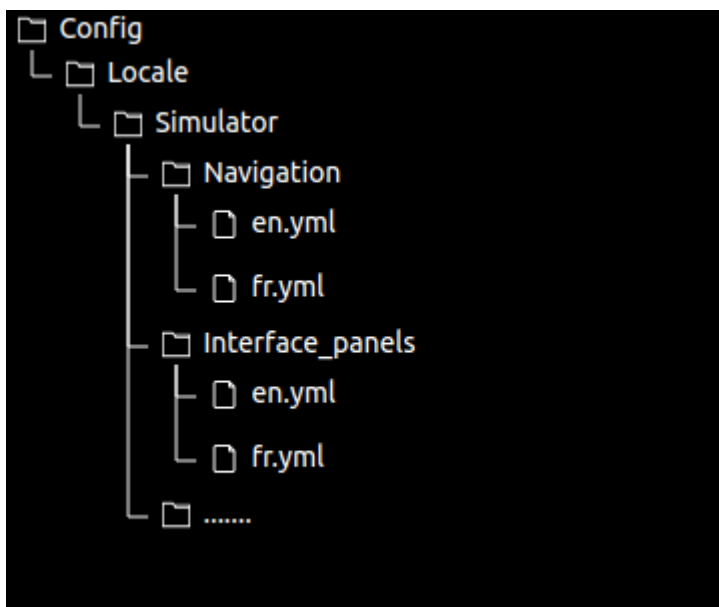
Translation Directory Structure:

Translation keys for Simulator will be stored in the form of **YAML** in **Config/locale/Simulator** folder including further nesting done according to the components in the simulator.

To avoid deep nesting of files and folders, namespaces will be used according to component names to separate translations of one component from another.

This means modularity will be provided using namespace and not using deep nesting of translation folders and files. Such approach is required as the majority of the Simulator code files do not have a large number of translations included.

General directory structure for the simulator can be designed as follows -



Internationalization of Simulator:

Internationalization in simulator includes I18n support in Rails side as well as JavaScript side.

Internationalization in Rails Side:

Simulator Interface written in **ERB** can be translated without any explicit configuration and by using Rails I18n translation function.

Localization of Simulator on Rails side include

- Localization of Bootstrap Navbar
- Localization of Dropdowns
- Localization of Heading and other relevant strings of draggable panels such as Properties panel, Circuit Elements panel etc
- Localization of other relevant strings coded on Rails side including tooltips in Simulator

Internationalization in Simulator JavaScript Codebase:

I will be using I18n-js as a yarn dependency to provide optimal support of I18n on the JavaScript side.

Generally I18n in simulator needs localization of following components -

Simulator Control Panels:

General examples of such draggable panels are -

- Circuit Elements Panel
- Timing Diagram Panel
- Properties Panel
- Edit Layout Panel

Panels are mainly composed of headings and content areas which are exported from JS files as html escaped strings and can be translated simply using a combination of translation function and template literals.

We can use template literals or concatenate html strings in variables in order to be able to use translation functions at appropriate positions.

Properties panels change their content area according to the element

selected from the Circuit Elements panel, Localization of Panels also includes analysis and localization of such dynamic content area of the panel.

JavaScript Prompts:

Simulator offers different features which can be seen in form of prompts exported from JS to the front end.

General examples of such prompts are:

- Insert SubCircuits
- Dec-Bin-Hex-Converter
- BooleanLogicTablePrompt

Most of the headings of Simulator prompts are directly coded in Simulator **.erb** file and hence can be translated using Rails I18n.

Content area of the prompts Majorly include techniques such as appending of HTML escaped strings on frontend, headings, data and components generated by some JavaScript logic etc. Translations functions can be used on relevant places in such strings by using basic JavaScript techniques such as template literals .

Components and Data Generated by JavaScript

The major challenge faced on the JavaScript side of Simulator is to provide localization of data and different components generated by complex JavaScript logic and stored in a variety of DataStructures. One of the example is Generation of truth table via Combinational Analysis

Before opting for localization in such section, deep analysis will be done and solutions will implemented after discussion with mentors

Other Sections to be localized

Some of the other relevant strings to be localized includes

- Error messages triggered while creating Circuits (Example : bit width error)

- Tooltips generated while hovering elements
- Report and Issue Form
- JavaScript alert messages

Internationalization of Interactive-Book (Jekyll Framework)

Table of Contents:

- [Synopsis](#)
- [Jekyll and Jekyll I18n Plugins](#)
- [Plugin Setup](#)
- [Plugin Configuration](#)
- [File Management and Directory Structure](#)
- [Internationalization of Interactive-Book](#)
- [Search Engine Optimization](#)

Synopsis:

Interactive book currently uses [just-the-docs](#) responsive theme with built in search. Jekyll doesn't provide native support for I18n, I18n support for Jekyll can be provided using multilingual plugins such as [Jekyll-Polyglot](#).

Internationalization of Interactive-Book basically includes -

- Providing I18n support in Jekyll.
- Configuration of basic attributes of I18n.
- Mechanism to serve localized versions of the site according to the current language selected by the user.
- Managing localized versions of the site in an efficient directory structure.
- Translation of site content and overriding Jekyll Just-the-Docs theme.
- Search Engine Optimization.

Jekyll and Jekyll I18n Plugins:

Use of multilingual plugins shall be decided foremost, as each plugin has its own translation management structure which tells us how our posts and pages should be organized and how to organize their metadata.

Popularly known Jekyll I18n plugins are [Jekyll-multiple-language plugin](#), [Jekyll-polyglot](#), [Jekyll-I18n](#)

After comparison between all the plugins ([See](#)), I will be using [Jekyll-Polyglot](#) for I18n support in Interactive-Book.

Why Jekyll Polyglot:

- Automated consistent site map:

Polyglot provides a consistent site map among all localized versions of the site.

Even if any post or page doesn't have its own translated version we will have its localized url for each supported language.

In case if we does not have a post in Chinese version still we will have localized version of url

- Automated Fallback support:

As Polyglot supports consistent sitemaps, localized versions of urls are generated for every post.

If a post is not present in current active language then polyglot serves the same post present in default language of the site.

- Automated localized permalink generation:

To localize permalink we do not need to define the local version of permalink explicitly for each language.

Polyglot generates localized urls for each supported language automated.

- Rich set of liquid tags

Basic attributes of I18n such as list of available languages, default language, current active language, I18n in templates are easy to use

via polyglot liquid tags.

- Text replacement of variables

In case when variables holding strings are used in site to display content, localization of such variable content can be done easily with polyglot `site.data` liquid tag.

Plugin Set up:

Polyglot plugin can be set up just by adding it to the bundler group in the gemfile. Any plugin listed in bundler group will get automatically activated even if it is not explicitly enabled in `_config.yml` file

```
group :Jekyll_plugins do
  gem 'Jekyll-Polyglot'
end
```

Plugin Configuration:

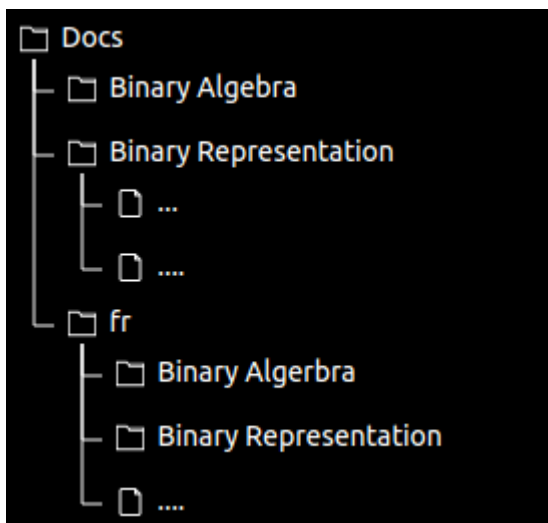
Polyglot Plugin provides different preference option to configure l18n as follows -

- Number of supported languages
- Default language and fallback language
- Option to exclude assets from being duplication in localized folders of the site
- Parallel or Serial language processing

File Management and Directory Structure:

Polyglot is automated, It follows the same organization of files as that of codebase and generates localized versions of the site.

A general demonstration of translation directory structure generated by Polyglot following the codebase structure is shown below



Internationalization of Interactive-Book:

Markdown files:

Major collections of markdown files are present in the **docs** folder along with other folders such as **_bibliography** and indexing pages for each section in **utils/chapters**.

Generally, translated versions of markdown files can be placed in respective language folders by following the designed directory structure and Polyglot will generate localized versions of site using such markdown files along with automated sitemap and fallback.

Some of the variable assigned attributes will be localized via creating **.yml** translation files for them

Translation for basic site variables:

Variables such as **site.title**, **site.descriptions** etc cannot be localized directly as such variables cannot hold multiple values in it.

Such attributes can be localized by modifying existing **layouts** and **includes** files which use such attributes.

Polyglot provides **site.data.attribute_name** which can access values for specific attributes stored in **_data** folder according to the current active language of the site.

Translation of Jekyll just-the-docs theme:

Non content strings hardcoded in the Jekyll theme needs to be translated and can be achieved by overriding the theme file which contains that string.

Generally themes are stored locally and can't be seen in projects root directory.

Overriding theme refers to bringing current required file of the theme into the project and feeding translations in them according to our need.

Localizing Date and Time:

To localize date and time we will declare '**date-time format strings**' for each language and use Polyglot **site.data.attribute_name** to display date time format according to the current active language of the site.

In _data folder

```
# en.yml
date_format: "%b %e, %Y"

# zh.yml
date_format: "%F"

can be accessed in layouts as site.data.date_format
```

Need for Language Switcher:

Language switcher can be easily designed by using polyglot liquid tags such as **site.languages**, **site.default_lang**, **site.active_lang** etc.

A Basic language switcher in Jekyll can be designed as follows -

,

```

{% for lang in site.languages %}
    {% if lang == site.active_lang %}
        {{ lang }}
    {% else %}
        {% if lang == site.default_lang %}
            <a href="{{ page.url }}">{{ lang }}</a>
        {% else %}
            <a href="/{{ lang }}{{ page.url }}">{{ lang }}</a>
        {% endif %}
    {% endif %}

{% endfor %}

```

Here we are fetching a list of supported languages and representing active language, default language and other available languages for the site.

we don't need `/en/` for default language so it is disabled for default language, this is done using Polyglot url relativization technique by just giving single space in anchor tag of default language

Search Engine Optimization:

Generally SEO can be optimized using html meta tags, using hreflang tag or by generating sitemaps using various plugins or gems.

Polyglot provides automated workflow for all the above SEO techniques with just one line of code ([reference](#)).

```

#header.html
{% I18n-Headers https://learn.circuitverse.org/ %}

```

Internationalization of CircuitVerse-Docs (DocsifyJs)

Table of Contents:

- [Synopsis](#)
- [Managing Multilingual Versions of CircuitVerse-Docs](#)
- [Translation Directory Structure](#)
- [Different components of Theme](#)
- [Internationalization of CircuitVerse Docs](#)

Synopsis:

CircuitVerseDocs is made using [DocsifyJs](#) documentation site generator and the site can be served locally by using a simple Python HTTP server.

Internationalization of themes provided by DocsifyJs does not require any external plugin support and can be done by building a basic mechanism for serving the right type of file according to the current active language of the site.

Managing Multilingual Versions of CircuitVerse Docs:

DocsifyJs [alias](#) feature leads in giving up two ways to manage multilingual version of site as follows

- Use different repositories for each language.
- Use the same repository with subfolders for each language.

However using different repositories can be hard to maintain. we will be using the same repository and store localized versions of the site by creating subfolders for each language.

Translation Directory Structure:

Directory structure for currently supported language :

Default language is currently supported language of CircuitVerseDocs i.e English.

Default language can be stored at the root of the serving directory or in the sub folder en according to our need. Both serve no change in url for the Default language.

```
Directory: /docs/  
or  
Directory: /docs/en/  
url: /#/
```

Directory structure for multiple languages:

localized versions of markdown files will be present in the subfolders resulting in more modular file structure with url being localized

```
Directory: /docs/fr/  
url: /#/fr/
```

Different Components of Theme:

Theme provided by DocsifyJs is generally divided into navbar, sidebar, content area.

Navbar:

Navbar is an essential part to represent the title of the site as well as the language selector. Navbar can be easily implemented using [LoadNavbar](#)

Generally Navbar in DocsifyJs expects content in markdown files and language switcher with appropriate links can be feeded in `_navbar.md`.

Sidebar:

Sidebar is an important component for navigating throughout the site and can be enabled using [LoadSidebar](#) with content in [_sidebar.md](#).

The Sidebar in Docsify is a static sidebar. We will be creating a localized Sidebar for each language and loading them according to the current active language of the site.

Content Area:

Content area is the container where markdown files are served.

As navigation to different pages is done using Sidebar, loading localized Sidebar leads to serving of appropriate localized markdown files in the content area.

Internationalization of CircuitVerse-Docs:

18n in Sidebar:

Users can navigate throughout the site using Sidebar. Sidebar contains navigation strings and appropriate URLs to markdown files.

Loading Sidebar with localized strings and urls helps us to serve appropriate a bunch of markdown files according to the current active language of the site.

We will be creating [_sidebar.md](#) for each language and fill appropriate URL targeting localized markdown files, in such a way we serve a bunch of markdown files which belong to the same language.

[Sidebar for English version](#)

```
#sidebar.md
[Home]()
```

[Sidebar for Russian version](#)

```
#sidebar.md
[дома](/ru/)
```

Content Area:

localized versions of markdown files can be stored in respective language folders.

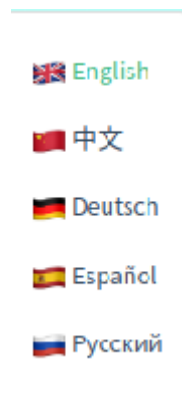
As the localized Sidebar will be loaded according to the current active language of the site ,navigation through sidebar will lead to serving of markdown files in the content area according to current active language.

Navbar:

Currently CircuitVerse-Docs doesn't load Navbar in theme, we will be loading Navbar to make a hold of language switcher in it.

Navbar is a simple markdown file holding language switcher written in the form of a list using markdown syntax. Language list will hold the Url for the subfolders of languages and DocsifyJs loads SideBar and markdown files present in corresponding subfolders

A demonstration of Language Switcher is as follows -



I18n in Mobile App (Flutter Dart)

Table of Contents:

- [Synopsis](#)
- [Configuration of I18n](#)
- [Internationalization of Mobile-App](#)
- [Language Switcher](#)

- [Persist User Locale](#)

Synopsis:

CircuitVerse Mobile-App is a cross platform Mobile-App written in Flutter. Flutter provides a rich set of built-in default widgets along with creation of flexible custom widgets.

Internationalization of flutter application includes internationalization of Flutter built-in default widgets and custom widgets.

Flutter provides localization of its default widgets in 78 different languages bundled in a single package named **flutter_localizations package**.

Localization of custom widgets involves usage of official **Intl package** which provides easy to use workflow to feed translation in Flutter application.

Configuration of I18n:

Currently CircuitVerse Mobile application uses basic setup of the official Flutter [Intl](#) package.

Current configuration of I18n uses a complex approach to feed translations in app and needs to be upgraded to a cleaner workflow.

Current workflow used requires CLI commands to generate files and needs to follow the steps given below -

- Create string resources for all the strings present in app
- Create AppLocalizations class and AppLocalizationsDelegate
- Generate **.arb** files from string resources
- Convert them into dart code by creating **messages_all.dart**
- Translations can be feed using **AppLocalizations.of(context).title**

Upgraded workflow will provide easy steps to provide optimal I18n support in Flutter and considered to be used

- Add **intl** package and **flutter_localizations** package with flag **generate:true**

- Create `l10n.yaml` describing path of translation directory and name of localization class.
- Create `.arb` files for providing translations.
- Run `flutter packages get` to trigger automated localization class generation.
- use `AppLocalizations.of(context).key_name` to provide translation

Upgraded workflow provides automated localization class generation and easy workflow to fill translations in existing translation files.

New translations when fed in existing translation files, get affected just by using `run flutter packages`.

Internationalization of Mobile-App:

Internationalization of Flutter Mobiles-App includes localization of Flutter built in widgets and custom widgets used throughout the app.

Flutter built in widgets:

To localize Flutter default widgets we will be using `flutter_localizations` package which provides localized versions of widgets in 78 different languages.

Package generally serves translations using three delegates

- `GlobalMaterialLocalizations.delegate`
Provides translations for all Flutter material widgets.
- `CupertinoLocalizations.delegate`
Provides translations for all Flutter cupertino widgets.
- `GlobalWidgetsLocalizations.delegate`
Provides global layout support to the widgets.

Some languages like Arabic are written from right to left, this delegates also provides optimal support in layout for languages written from left to right or from right to left.

Flutter Custom Widgets:

Flutter Custom Widgets are classes created to serve specific purposes.

Localization of Custom Widgets requires creation of a custom localization class and its corresponding delegates.

Intl provides automated way to create custom localization class and its delegate just by adding following line in configuration file -

output-localization-file: `app_localizations.dart`

It generates `app_localizations.dart` holding `AppLocalizations()` and `AppLocalizationsDelegate()` which can be easily imported in app using `AppLocalizations.delegate`

Localizing Flutter Static Data:

Most of the static strings lie in the Views folder and can be abstracted into `.arb` files to localize them.

Conversion of translations files into dart code is done by Intl in an automated way.

Localizing Flutter Dynamic Data:

CircuitVerse Mobile-App fetches data from CircuitVerse code base via CircuitVerse API in form of JSON.

Such data rendered on mobile app needs to be fetched in localized form of JSON response, according to the current active language of the mobile app

As a solution we can allow users to set a locale via `url query parameter (?locale=fr)` on CircuitVerse main platform and deliver a localized form of response to the mobile app according to the current active locale set on the mobile app.

Language Switcher:

Language switcher is an Important UI component to be designed and will be implemented in the AppBar of the mobile app.

Flutter provides [`DropDownButton`](#) property which helps in creating a button

along with a list of selected items easily.

Flutter provides [Icons class](#) to set the neat language Icon for the DropdownButton.

Items in dropdown list can have many languages to display, As a solution I will be creating a custom class which will hold the list of all the supported languages with their locale value and language name of locales and they can be mapped to the list of select items for DropdownButton.

DropdownButton is used with [onChanged](#) callback which sets selected values for locale.

Persist User Locale:

Persisting locales set by users helps the user to view the content in the same language every time the user opens the app.

Persisting of locales will be done using the [shared_preferences](#) package which allows easy storage and retrieval of small amounts of data such as user locale.

Project Plan

End goal of the project is to deliver I18n infrastructure for all the CircuitVerse platforms.

Major part of the project includes localization of Rails platform, Simulator and Mobile app.

As Translations in such platforms need to be constructed in the form of key value pairs, I will be giving sensitive focus in naming of keys, Translation Directory structure, along with concise mechanism and documentation for I18n.

Interactive-Book and Circuitverse-Docs are static sites following the markdown approach. Here we need translations in the form of key value pairs in a less amount and the majority of I18n workflow is automated.

In this case, I will be giving more focus on the implementation of I18n along with creation of flexible directory structure, so that feeding of translations in them can be done in an easy way for any type of language.

Initially, I will start analysing the codebase of all the platforms in depth and will actively clear doubts and issues with respective mentors during the pre GSoC period. Along with this I will be active full time on slack channel and I will frequently research and discuss with mentors to get optimal solutions for the challenges faced.

Generally I will learn and code throughout the week and on weekends I will opt for code quality improvements, documentations and pull requests. Along with this I will frequently report to the mentors about the work completed every week.

I will be working for I18n in Rails and Simulator in both the phases of GSoC. Although many of the sections of Simulator can be localized using basic logic, I will not directly opt for translation in some of the JavaScript based sections such as Simulator Controls (Draggable Panels , Prompts etc).

To localize such sections I will be planning, discussing and finalizing solutions with mentors before the implementation step. In this way Simulator will be localized in a procedure of Analysis and implementation for generating best possible results.

In phase one major part of Rails and Simulator Interface will be localized along with internationalization of CircuitVerse-Docs and will be delivered.

In phase two I will be completing Localization of Interactive-Book, Mobile App and along with localization of remaining portions of Rails and Simulator. In this way I will be giving major focus on Simulator and Rails along with other three platforms.

Project Plan - Preliminary Plan :

Pre GSoC period [4 weeks: April 13 - May 17]

I will be using this time for in-depth analysis of the codebases of the platforms and getting familiar with strings encoded in different sections of the codebase.

Major time I will spend on the JavaScript portion of Simulator and will experiment with the codebase to get a better understanding of Interface and features provided by Simulator.

In pre GSoC period I will be majorly focussing on -
Localization of Model Data in Rails.

Localization of dynamic data of Mobile-App delivered by CircuitVerse API.
Localization of Simulator Controls such as Prompts, Panels and other complex sections of Simulator.

Community Bonding Period [3 weeks: May 17 - June 7]

During this period, I will be discussing and finalizing concrete approaches to complete the project with the mentors and I will clear my doubts regarding project implementation and other off project topics.

Along with this I will be setting up codebases of all platforms and different tools and plugins required for development before proceeding in the coding period.

Coding Period:

Phase 1 [5 weeks: June 7 - July 11]

Week 1 - Week 2

- Proceeding for Internationalization of Rails Platform and Simulator with appropriate Development Environment and Key Management Tools setten up.
- Configure I18n Module, Create Language Switcher.
- Localization of Rails and Simulator Interface (Static data only for better naming convention and translation directory design).

- Localization for strings popped from different gems.(Devise, Paginator).
- Imp
- rovements in Naming of Keys, Translation Directory Structure, Fixing Translation Missing Errors (If any), Removal of Unused keys (If any).
- Push Pull Requests.Overview of data which is to be translated in upcoming week

Week 3 - Week 4

- Localization of Model data in Rails including validation messages.
- Integrate I18n-js with the platform.
- Localization of Simulator UI Dialogues and Draggable Panels (Major ones but not all as model localization might take time or need some improvements).
- Internationalization of CircuitVerse-Docs.
- Remaining improvements in previous work (If any) and analysis of data which is to be translated in the upcoming week.
- Push/Modify Pull Requests.

Week 5

- Continuing localization of Simulator UI Dialogues and Draggable Panels (As much as possible)
- Major improvements before phase one evaluations
- Push Pull Requests
- Analysis of data which is to be translated in the upcoming week.

GSoC phase one evaluations [July 12 - July 16]

Before phase one evaluations I will be completing major I18n tasks in Rails Platform and Simulator excluding some complex sections to get more time for better analysis and concrete implementation in those areas.

Key deliverables:

In Rails:

- Views
- Models
- Forms
- Controllers
- I18n in gems (Devise, Paginator etc)
- Translations for other Relevant Strings of Rails platforms.

In Simulator:

- Integrating I18n-js with Rails Platform
- Complete Localization of Simulator Interface written in ERB
- Localization of Panels visible on the Canvas and others if possible.
- Translations for other Relevant Sections of Simulator.

In CircuitVerse-Docs:

- Complete Internationalization of CircuitVerse-Docs

Phase 2 [4 weeks: July 17 - August 15]

Week 1 - Week 2

- Proceeding Internationalization in Interactive-Book and continuing localization in Simulator
- Configure I18n and create Language Switcher in Interactive-Book.
- Create Infrastructure for I18n and localize different sections of the site along with proper Implementation of SEO.
- Testing, identifying, fixing potential issues (If any)
- Completing remaining localization in Rails and Simulator and other relevant sections.
- Push/ Modify Pull Requests

Week 3 - Week 4

- Configure I18n, Create Language Switcher in Mobile App
- Extraction and localization of static data in `.arb` files
- Implementing solutions to localize dynamic data in Mobile-App delivered by CircuitVerse API (Requires coding on Rails side).
- Using remaining time for improvements in completed code in Rails, Simulator, and other completed sections .
- Implementation of some additional features (If required).

Code submission and GSoC final evaluations [1 Week: August 23 - August 30]

All the CircuitVerse platforms will be localized and delivered as committed.

Additional information about timeline

Project timeline is tentative and designed roughly according to the initial research done so far. I will try my best to keep the progress up according to the timeline.

After in-depth analysis of the codebase, In case if there is any change in timeline, I will be informing mentors about it during the community bonding period.

Major Milestones

- Complete Infrastructure for Internationalization of Rails Platform
- Complete Infrastructure for Internationalization of Simulator Platform
- Complete Infrastructure for Internationalization of Mobile-App
- Complete Infrastructure for Internationalization of Interactive-Book
- Complete Infrastructure for Internationalization of CircuitVerse-Docs

Additional Information

My Development Environment comprises **Ubuntu 20.4** and **Visual Studio Code** as a code editor. Along with this I use multiple extensions such as **RuboCop**, **liquid**, **markdownlint** and many more to provide quality code.

For Android Development, I prefer **Android studio** along with common Flutter extensions for effective development in CircuitVerse Mobile App.

I believe that I am a perfect candidate to complete this project, I am very familiar with CircuitVerse Primary codebase and I have done the majority of contributions in Rails and Simulator.

Most of the contributions done by me were feature oriented. (Group Invitation System, Site Wide Announcements, Generate Circuit from Boolean Function and others).

Development of such features assisted me in providing a brief overview of the codebase, Now I am familiar with the major parts of the codebase and have a good understanding of the codebase.

I have also completed my GSoC qualification task before time and I am confident enough to give the best possible I18n infrastructure before the deadline.

GSoC qualification task: [See](#)

I have completed the majority of the homework on this project. During this, I shared my approach for I18n and got them reviewed by the mentors.

- [GistA](#)
- [GistB](#)
- [GistC](#)
- [GistD](#)
- [GistE](#)

Post GSoC Plans

Major challenge faced after feeding translations for different language will be lack of flexible UI of all the platforms. After GSoC I will be actively contributing to CircuitVerse to help in making UI flexible and adaptable to the translations for different languages.

Mentors

[Aboobacker MK](#), [Nitish Aggarwal](#)